

OpenCoderRank: Personalized Technical Assessments with Generative AI

Hridoy Sankar Dutta*

Deakin University

GIFT City, India

hridoy.dutta@deakin.edu.au

Swati Kumari

Deakin University

GIFT City, India

swati.kumari@deakin.edu.au

Sana Ansari

S.P. Jain Institute of Management and Research

Mumbai, India

sana.ansari@spjimr.org

Shounak Ravi Bhalerao

Deakin University

GIFT City, India

shounakbhalerao777@gmail.com

Abstract

Organizations and educational institutions use time-bound assessment tasks to evaluate coding and problem-solving skills. These assessments measure not only the correctness of the solutions, but also their efficiency. Problem setters (educator/interviewer) are responsible for crafting these challenges, carefully balancing difficulty and relevance to create meaningful evaluation experiences. Conversely, problem solvers (student/interviewee) apply critical and logical thinking to arrive at correct solutions. In the era of Large Language Models (LLMs), LLMs assist problem setters in generating diverse and challenging questions, but they can undermine assessment integrity for problem solvers by providing easy access to solutions. We introduce OpenCoderRank a lightweight, self-hosted platform that emulates real-world timed technical assessments in resource-constrained environments. OpenCoderRank is intentionally model-agnostic: it facilitates the creation, deployment and automatic grading of problems while offering fine-grained control over time limits, input-output pairs and execution constraints. OpenCoderRank is evaluated using two methods: 1. *BERTScore*, 2. LLM evaluation. Our findings indicate that OpenCoderRank connects problem setters and solvers by supporting time-constrained preparation and self-hosted, customizable assessments in resource-constrained settings.

CCS Concepts

• **Information systems** → Users and interactive retrieval; • **Computing methodologies** → Knowledge representation and reasoning; • **Human-centered computing** → Interaction design.

Keywords

Education, Large Language models, Exercise generation, generative AI, LLMs, test cases

ACM Reference Format:

Hridoy Sankar Dutta, Sana Ansari, Swati Kumari, and Shounak Ravi Bhalerao. 2026. OpenCoderRank: Personalized Technical Assessments with Generative AI. In *Proceedings of The First Workshop on Synthetic Content in Information*



This work is licensed under a Creative Commons Attribution 4.0 International License. *SynthIR@SIGIR 2026, Melbourne, VIC, Australia*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

Retrieval Ecosystems (SynthIR@SIGIR 2026), July 24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Introduction

With the rise in Large Language Models (LLMs), they have become valuable tools for problem setters, helping them create a wide variety of complex and well-structured questions more efficiently [5, 15]. However, problem solvers increasingly utilize LLMs, thereby creating challenges for problem setters in maintaining the fairness of technical assessments through readily quick code generation and creating an uneven playing field [1, 6, 10]. The technical assessments often include time-bound assessments that test both correctness and performance against tests. For example, in academic setting, to help students measure their problem-solving ability, universities often tie up with cost-intensive third-party platforms or outsource assessment services to external vendors. Without a controlled environment, students can easily open multiple tabs or access external tools during assessments, undermining the validity of test results. These issues cannot be mitigated effectively unless the assessment platform enforces restrictions to ensure legitimate assessment practices such as disabling copy-paste, enforcing full screen mode, etc.

To address these challenges, we present OpenCoderRank (Source Code¹, Video tutorial² and Tool Prototype³), a platform designed to host assessment tasks that simulate industry and academic style challenges to reduce the gap between problem setters and problem solvers. Figure 1 shows an overview of OpenCoderRank with key features labeled. OpenCoderRank aims to answer two research questions.

- **RQ1:** How can a self-hosted assessment platform improve pedagogy in a resource-constrained environment?
- **RQ2:** How can a self-hosted assessment platform maximize its effectiveness in the era of LLMs?

We use the terms *interviewers*, *educators* and *recruiters* interchangeably to refer to problem setters on the OpenCoderRank platform, and the terms *interviewees*, *students* and *participants* to refer to problem solvers.

¹<https://anonymous.4open.science/r/OpenCoderRank-D13E/>

²https://anonymous.4open.science/r/OpenCoderRank-D13E/OpenCoderRank_video.mp4

³<https://opencoderrank.onrender.com/>

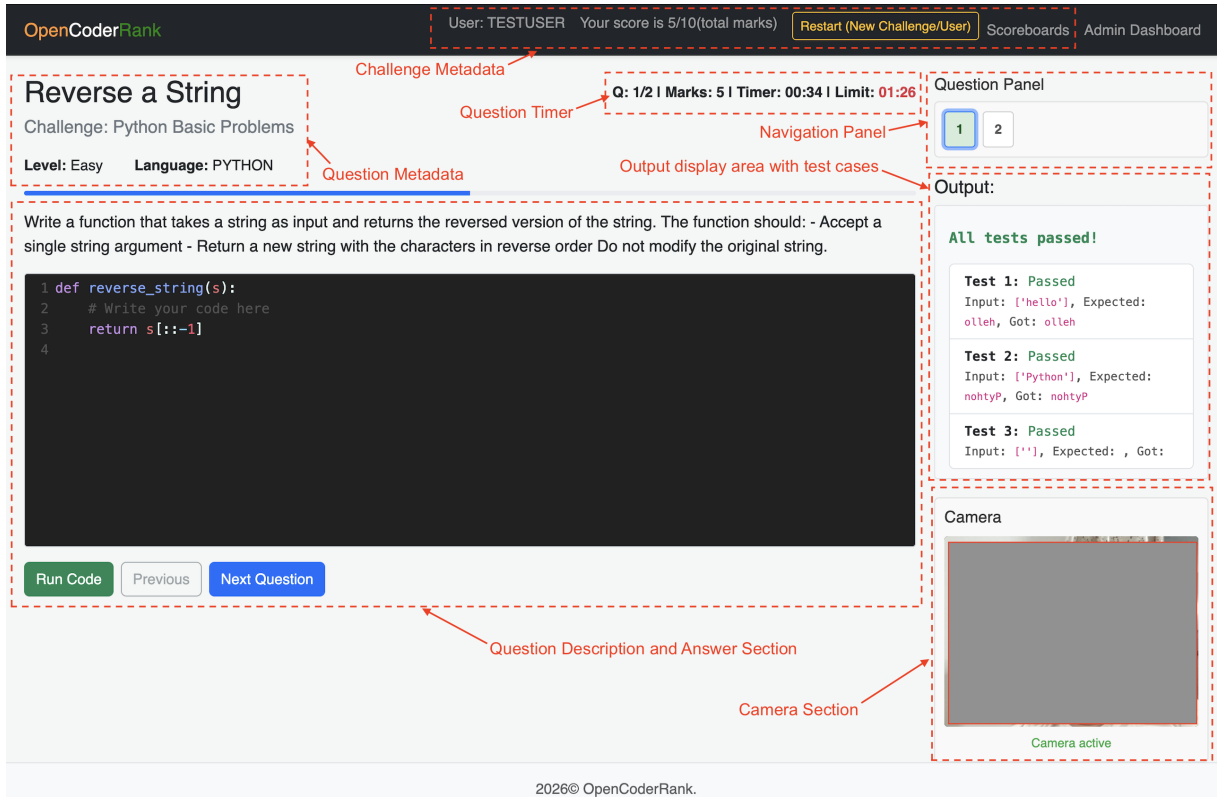


Figure 1: Overview of the OpenCoderRank interface with key features labeled.

Related Work

Educators and interviewers around the world are increasingly using LLMs to create innovative methods to teach programming and develop the technical skills of students [2, 3, 19]. For example, [13] introduced BugSpotter, which uses LLMs to auto-generate buggy Python code exercises that help students develop proficiency in bug localization by designing failing test cases. [11] developed CodeHelp, an LLM-powered tool that provides on-demand programming assistance without directly revealing solutions. [9] designed live oral coding interviews to evaluate student proficiency based on take-home work. Besides technical assessments, LLMs have also been explored for other technical educational purposes such as reducing educator workload [4, 7, 14], teaching students debugging strategies for programming [8, 17], etc. We provide a detailed comparison of OpenCoderRank with existing tools in the literature, as summarized in Table 1.

Our study: While the above mentioned tools focus on enhancing learning and developing skills, they do not fully address critical challenges in technical assessments in the current scenario. Motivation for industry is different, recruiters primarily aim to test whether students possess fundamental problem-solving skills and can think critically under pressure. To evaluate this, companies often rely on targeted programming tasks designed to test whether students arrive at the correct output efficiently. This outcome-focused evaluation contrasts with many existing educational tools that prioritize

guiding the student through the learning process rather than simulating real-world test conditions. Hence, there is a gap between what students practice in academic settings and what they are expected to demonstrate during industry hiring. OpenCoderRank

Table 1: Comparison of OpenCoderRank with existing tools in the literature

Feature / Tool	[13]	[11]	[9]	[5]	Ours
LLM Question Generation	✓	✗	✗	✓	✓
Automated Judging / Scoring	✗	✗	✗	✓	✓
Self-Hosted / Local Deployment	✗	✗	✗	✗	✓
Integrity Controls (Fullscreen, Copy-Paste Lock)	✗	✗	✗	✗	✓
Randomized Question Sequencing	✗	✗	✗	✗	✓
Programming/MCQ modes	✗	✓	✓	✗	✓
Camera Feature	✗	✗	✗	✗	✓
Resource-Constrained Friendly	✗	✗	✗	✗	✓

bridges the gap between problem-setters and problem-solvers by combining the flexibility of learning platforms with the rigor of industry-style assessments. It ensures academic integrity, rapid setup and deployment and a framework for creating customized challenges.

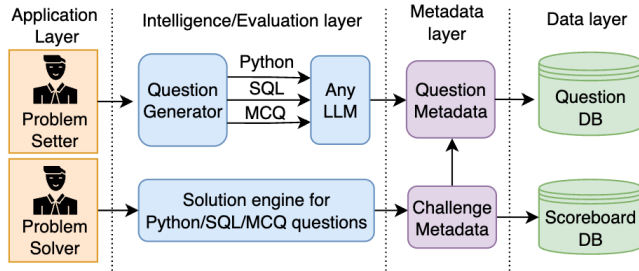


Figure 2: High-level pipeline of OpenCoderRank

OpenCoderRank Overview

In this section, we will discuss key technical details of OpenCoderRank. The overall pipeline of OpenCoderRank is presented in Figure 2 spanning application, intelligence/evaluation, metadata and data layers. A problem setter creates questions via a generator that can leverage any LLM for Python, SQL or MCQ tasks, while a solution engine evaluates submissions from problem solvers. Generated questions and challenges are stored with relevant metadata in dedicated databases. OpenCoderRank is built with the Python Flask⁴ framework and uses SQLite⁵ as the database.

Key Features

OpenCoderRank offers a comprehensive set of features for both problem setters and problem solvers. Problem setters can create, edit and manage problems and problem solvers can engage effectively with the problems. Each problem includes

- **Challenge details:** Related *Challenge ID* for the problem.
- **Problem details:** *Title, description and level (easy/medium/hard)* of the problem.
- **Language:** *mcq* for MCQ questions, *python/sql* for coding questions.
- **Input/output specifications:** *Valid/invalid options* and *Correct answer* for MCQ questions, *starter code* and *test cases* for coding-based assessments, *time limit* and *points* for both MCQ and coding-based assessments.
- **Remarks:** Any *remarks/hints* related to the problem.

OpenCoderRank also supports an “Camera” feature that allows problem setters to guide problem solvers during assignment solving.

Question Generator and Judging

Inspired by Chain-of-Thought reasoning [16], OpenCoderRank encourages more structured and transparent reasoning in the creation and evaluation of technical assessments. OpenCoderRank is developed primarily as an assessment platform. Problem setters can design their own set of questions or generate one question to create new, increasingly difficult questions using the AI prompt. OpenCoderRank supports seamless, web-based question generation that is fully embedded into the workflow, removing the need for manual copy-and-paste of prompts as shown in Figure 3. For educators from non-technical backgrounds, we additionally provide a manual

question creation interface specifically for MCQ questions. OpenCoderRank supports automated judging of MCQ questions and supports two programming languages - Python and SQL.

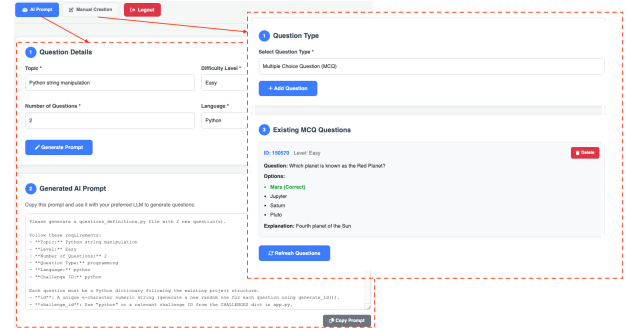


Figure 3: Question Generator with the prompt to generate questions on any LLM (a longer and more detailed version of the question generator prompt is present in the GitHub repository).

Rapid Setup and Deployment

One of the key strengths of OpenCoderRank is its simplicity and speed of deployment. The platform is built for quick setup and can be self-hosted in just two straightforward steps. The other alternatives are to host it on cloud-based platforms⁶ for broader access or share it through tunneling tools⁷ to run it with minimal overhead. This makes it especially valuable for organizers who need a fast, flexible and reliable solution for events with minimal configuration. OpenCoderRank includes Docker support that allows users to containerize the entire application and deploy it consistently across different systems. This makes it especially valuable for problem setters who need a fast, flexible and reliable solution for events with minimal configuration.

Evaluation

OpenCoderRank is model-agnostic and does not depend on any specific LLM. We therefore evaluate the quality of assessment content hosted on the platform, rather than the performance of any particular generation model, similar to the approach followed by Molavi et al. [12]. We assess the quality of OpenCoderRank-generated technical assessment questions using a combination of embedding-based semantic similarity and model-based automatic evaluation. Specifically, we employ BERTScore [18] to measure semantic alignment between generated questions and expert-authored problems from LeetCode and HackerRank. BERTScore leverages contextual embeddings from pretrained language models to enable robust comparison beyond surface-level lexical overlap. We also adopt an LLM-as-a-Judge evaluation framework following recent work on automated model-based evaluation [5, 15], in which a state-of-the-art large language model is prompted as an expert computer science educator to assess each generated question along four dimensions: *clarity* (readability and unambiguity), *correctness* (technical validity

⁴<https://flask.palletsprojects.com/en/stable/>

⁵<https://sqlite.org/index.html>

⁶<https://www.netlify.com/>

⁷<https://pinggy.io/>

and logical consistency), *pedagogical usefulness* (instructional and assessment value) and *difficulty alignment* (appropriateness relative to the intended difficulty level).

We generate the questions using GPT-4o and evaluate them using the Gemini 3 Flash model⁸ within our automated generation and evaluation pipeline. From an evaluation of 100 technical assessment questions sourced from LeetCode and HackerRank, we observe that OpenCoderRank produces high-quality and pedagogically meaningful content that closely aligns with expert-authored problems, as summarized in Table 2. We use a different model for evaluation to reduce bias, since models tend to favor outputs similar to their own style. We observe that semantic similarity analysis using BERTScore yields an F1 score of 0.79, indicating substantial alignment with reference problems. To further assess question quality, as mentioned

Table 2: Evaluation of OpenCoderRank-generated assessment questions compared to expert-authored problems from LeetCode and HackerRank.

Evaluation Aspect	Score	Mechanism
Semantic Similarity (BERTScore)	0.79/1	vs. expert-authored
Question Clarity	4.98/5	LLM-as-a-Judge
Question Correctness	4.93/5	LLM-as-a-Judge
Pedagogical Usefulness	4.92/5	LLM-as-a-Judge
Difficulty Alignment	4.39/5	LLM-as-a-Judge

above, we employ an LLM-as-a-Judge framework. This evaluation confirms the strong quality of the generated questions, with high average scores for clarity (4.98/5) and correctness (4.93/5), reflecting well-structured and technically sound problem statements. Pedagogical usefulness achieves a score of 4.92/5, demonstrating that the questions are well suited for instructional and assessment purposes. In addition, the difficulty alignment score (4.39/5) indicates that the generated problems generally match their intended difficulty levels.

Answering the research questions

RQ1: Pedagogical Implications

OpenCoderRank is designed to enhance learning outcomes across a variety of teaching and evaluation contexts. Below are several pedagogical use cases where OpenCoderRank can be effectively used:

- **Coding Competitions:** OpenCoderRank provides a unique environment for hosting intra- or inter-college coding competitions. Various coding clubs within academic institutions can also deploy it locally for quick peer learning exercises. With features like fast setup and real-time evaluation, the platform enables organizers to run contests even in resource-constrained environments with zero setup costs.
- **Classroom Integration:** Although OpenCoderRank is primarily focused on coding and programming exercises, the platform also supports MCQs, making it suitable for quick assessments or pre/post-lecture quizzes. Educators can incorporate OpenCoderRank into regular classroom teaching as part of time-constrained lab sessions or live quizzes.

⁸<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-flash>

- **Academic Lab/Startup Hiring:** The easy setup of OpenCoderRank allows academic research labs and startups from various domains to conduct technical assessments for hiring assistants or interns. Custom challenges aligned with specific focus areas (e.g., MCQs on domain knowledge or relevant programming tasks) can be used to evaluate candidate abilities prior to subsequent rounds of the interview process.
- **One-to-One/One-Vs-One Challenges:** The platform supports personalized one-to-one coding challenges, which are especially useful in mentoring sessions or during office hours. Educators can assign individualized tasks and provide real-time feedback to create a focused learning experience.

RQ2: Ensuring Integrity in the LLM Era:

The rise of LLMs poses new challenges for maintaining integrity in technical assessments. To encounter this, OpenCoderRank implements several strategies to mitigate this risk:

- **Full-screen mode enforcement:** OpenCoderRank is designed to run only in full-screen mode. When participants click the “Start Test” button, a prompt informs them that the assessment must be taken in full-screen mode to minimize distractions. If a participant exits or disables full-screen mode five times, the test is automatically submitted.
- **Disabling copy-paste:** OpenCoderRank promotes legitimate assessment practices, such as writing code without relying on external tools by preventing direct copying from LLMs or external sources.
- **Random sequencing of questions:** Questions are presented in a randomized order making it difficult to rely on neighboring screens during assessments.

Conclusion and Future Work

This paper presents OpenCoderRank, an LLM-powered platform designed to overcome the limitations of traditional technical assessments. OpenCoderRank serves as a pedagogical bridge between problem setters and problem solvers, enabling students to practice under timed conditions and tackle unfamiliar problems, while empowering problem setters to self-host and customize assessments. By ensuring integrity for technical assessments, OpenCoderRank adapts to modern evaluation challenges while preserving fairness in the era of LLMs.

OpenCoderRank represents a step forward in personalized education, offering customized creation and evaluation for designing and evaluating technical assessments and by setting the stage for future innovations in adaptive learning methodologies. In future work, we plan to incorporate more advanced user-profiling signals such as skill trajectories, problem-solving behavior patterns and difficulty adaptation curves to provide deeper insight and more personalized assessment experiences. The range of assessment types supported by OpenCoderRank will continue to expand as well. We also aim to address key technical challenges to safeguard integrity in the LLM era that uphold assessment integrity in OpenCoderRank by focusing on robust anti-cheating strategies and reliable verification mechanisms.

References

- [1] BHUSHAN, S., THOMAS, D. R., BORCHERS, C., RAGHUVANSHI, I., ABBODD, R., GATZ, E., GUPTA, S., AND KOEDINGER, K. Detecting llm-generated short answers and effects on learner performance. *arXiv preprint arXiv:2506.17196* (2025).
- [2] CHU, Z., WANG, S., XIE, J., ZHU, T., YAN, Y., YE, J., ZHONG, A., HU, X., LIANG, J., YU, P. S., ET AL. Llm agents for education: Advances and applications. *arXiv preprint arXiv:2503.11733* (2025).
- [3] CHU, Z., XIE, J., WANG, S., WANG, Z., AND WEN, Q. Uniedu: A unified language and vision assistant for education applications. *arXiv preprint arXiv:2503.20701* (2025).
- [4] DEL CARPIO GUTIERREZ, A., DENNY, P., AND LUXTON-REILLY, A. Evaluating automatically generated contextualised programming exercises. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (2024), pp. 289–295.
- [5] DESMOND, M., ASHTORAB, Z., GEYER, W., DALY, E. M., COOPER, M. S., PAN, Q., NAIR, R., WAGNER, N., AND PEDAPATI, T. Evalassist: Llm-as-a-judge simplified. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2025), vol. 39, pp. 29637–29639.
- [6] FLECKENSTEIN, J., MEYER, J., JANSEN, T., KELLER, S. D., KÖLLER, O., AND MÖLLER, J. Do teachers spot ai? evaluating the detectability of ai-generated texts among student essays. *Computers and Education: Artificial Intelligence* 6 (2024), 100209.
- [7] JORDAN, M., LY, K., AND SOOSAI RAJ, A. G. Need a programming exercise generated in your native language? chatgpt’s got your back: Automatic generation of non-english programming exercises using openai gpt-3.5. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (2024), pp. 618–624.
- [8] KAFAI, Y. B., DELIEMA, D., FIELDS, D. A., LEWANDOWSKI, G., AND LEWIS, C. Rethinking debugging as productive failure for cs education. In *Proceedings of the 50th ACM technical symposium on computer science education* (2019), pp. 169–170.
- [9] KANNAM, S., YANG, Y., DHARM, A., AND LIN, K. Code interviews: Design and evaluation of a more authentic assessment for introductory programming assignments. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (2025), pp. 554–560.
- [10] KOIKE, R., KANEKO, M., AND OKAZAKI, N. Outfox: Llm-generated essay detection through in-context learning with adversarially generated examples. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), vol. 38, pp. 21258–21266.
- [11] LIFFITON, M., SHEESE, B. E., SAVELKA, J., AND DENNY, P. Codehelp: Using large language models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research* (2023), pp. 1–11.
- [12] MOLAVI, M., TAVAKOLI, M., MOEIN, M., FARAJI, A., AND KISMIHÓK, G. Llm-driven personalized answer generation and evaluation. In *International Conference on Artificial Intelligence in Education* (2025), Springer, pp. 187–194.
- [13] PĂDUREAN, V.-A., DENNY, P., AND SINGLA, A. Automated generation of code debugging exercises. *arXiv e-prints* (2024), arXiv–2411.
- [14] PHUNG, T., PĂDUREAN, V.-A., CAMBRONERO, J., GULWANI, S., KOHN, T., MAJUMDAR, R., SINGLA, A., AND SOARES, G. Generative ai for programming education: benchmarking chatgpt, gpt-4, and human tutors. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 2* (2023), pp. 41–42.
- [15] VERGA, P., HOFSTATTER, S., ALTHAMMER, S., SU, Y., PIKTUS, A., ARKHANGORODSKY, A., XU, M., WHITE, N., AND LEWIS, P. Replacing judges with juries: Evaluating llm generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796* (2024).
- [16] WEI, J., WANG, X., SCHUURMANS, D., BOSMA, M., XIA, F., CHI, E., LE, Q. V., ZHOU, D., ET AL. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [17] WHALLEY, J., SETTLE, A., AND LUXTON-REILLY, A. Novice reflections on debugging. In *Proceedings of the 52nd ACM technical symposium on computer science education* (2021), pp. 73–79.
- [18] ZHANG, T., KISHORE, V., WU, F., WEINBERGER, K. Q., AND ARTZI, Y. Bertscore: Evaluating text generation with bert. In *International Conference on Learning Representations (ICLR)* (2020).
- [19] ZHANG, Z., ZHANG-LI, D., YU, J., GONG, L., ZHOU, J., HAO, Z., JIANG, J., CAO, J., LIU, H., LIU, Z., ET AL. Simulating classroom education with llm-empowered agents. *arXiv preprint arXiv:2406.19226* (2024).